

End-to-End Deep Learning Workflow

Dr. Parham Kebria
Senior Research Scientist
parhamkebria@ieee.org

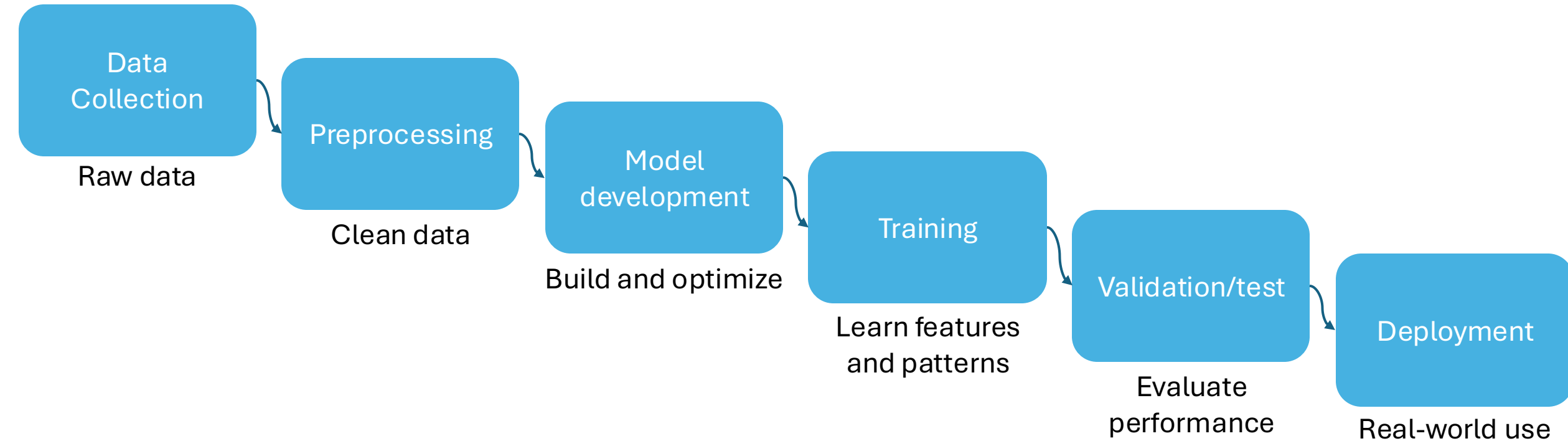
Advanced Study Institute: Artificial Intelligence for Disaster Management
Orlando, November 17 – 25, 2025



*This activity
is supported by:*

The NATO Science for Peace
and Security Programme

What is a Deep Learning Workflow?



Why an End-to-End Pipeline Matters

- Real-world data is messy, not like curated benchmarks
- Most project times spent on *data* and *validation*
- Deployment ensures the model's value is realized

An End-to-End example:

Drone detecting flood-affected areas

→ insights delivered via mobile dashboard

→ response actions to be taken based on the insights

→ Data Sources Identified & Collected



→ Data Sources Identified & Collected



→ Data Preprocessing & Augmentation for Training



→ Data Sources Identified & Collected

→ Data Preprocessing & Augmentation for Training

→ Design & Develop the Model Architecture



→ Data Sources Identified & Collected

→ Data Preprocessing & Augmentation for Training

→ Design & Develop the Model Architecture

→ Model Training & Validation → Model Optimization



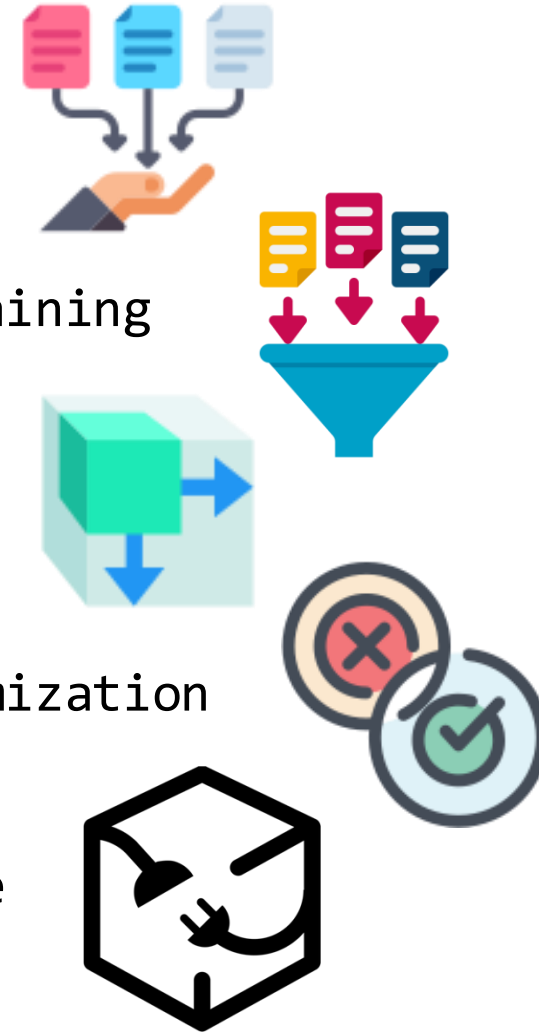
→ Data Sources Identified & Collected

→ Data Preprocessing & Augmentation for Training

→ Design & Develop the Model Architecture

→ Model Training & Validation → Model Optimization

→ Deployment of the Final Model in Practice



Sources:

- Sentinel-2, Landsat, Planet Labs, Kaggle, etc.

Sources:

- Sentinel-2, Landsat, Planet Labs, Kaggle, etc.

Challenges:

- Cloud cover, varying lighting, perspective distortion
- Different resolutions and noisy/incorrect labels

Sources:

- Sentinel-2, Landsat, Planet Labs, Kaggle, etc.

Challenges:

- Cloud cover, varying lighting, perspective distortion
- Different resolutions and noisy/incorrect labels

Labeling Strategies:

- Manual annotation (slow but accurate)
- Crowdsourcing (Amazon MTurk, LabelBox)
- Semi-supervised or weakly labeled approaches

Sources:

- Sentinel-2, Landsat, Planet Labs, Kaggle, etc.

Challenges:

- Cloud cover, varying lighting, perspective distortion
- Different resolutions and noisy/incorrect labels

Labeling Strategies:

- Manual annotation (slow but accurate)
- Crowdsourcing (Amazon MTurk, LabelBox)
- Semi-supervised or weakly labeled approaches

Tip:

Always inspect a subset of
your data visually

Sources:

- Sentinel-2, Landsat, Planet Labs, Kaggle, etc.

Challenges:

- Cloud cover, varying lighting, perspective distortion
- Different resolutions and noisy/incorrect labels

Labeling Strategies:

- Manual annotation (slow but accurate)
- Crowdsourcing (Amazon MTurk, LabelBox)
- Semi-supervised or weakly labeled approaches

Tip:

Always inspect a subset of
your data visually

Handling Noisy Labels

Problem:

- Mislabeling
- Unclear boundaries
- Class imbalance

Solution:

- Cross-check with multiple annotators
- Use polygon-based labeling
- Oversampling/class weights

Preprocessing Essentials:

- Resize all images to a common input shape
- Normalize pixel values ([0,1] or [-1,1])
- Split dataset → Train / Val / Test

Common Augmentations:

<i>Technique</i>	<i>Example</i>	<i>Purpose</i>
Flip/Rotate	90°, 180°	Orientation invariance
Random Crop	Random patch	Simulate zooming
Color Jitter	Vary brightness/contrast	Lighting robustness
Gaussian Noise	Add small noise	Robustness to sensor noise



```
import torchvision.transforms as T

transform = T.Compose([
    T.Resize((224, 224)),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406],
                 std=[0.229, 0.224, 0.225]))
```



```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=15,
    horizontal_flip=True,
    zoom_range=0.1)
```



Deployment Options:

<i>Platform</i>	<i>Use Case</i>	<i>Tools</i>
Mobile App	On-device detection	TensorFlow Lite / Core ML
Drone	Real-time edge inference	NVIDIA Jetson / ONNX
Web API	REST-based model serving	FastAPI / Flask + TorchServe



This Session:

- ❖ Deep learning = pipeline, not just model
- ❖ Data quality >> Model complexity
- ❖ Always visualize and validate at each step
- ❖ Deployment ensures real-world impact



Previous sessions:

- ✓ Image Classification Fundamentals
- ✓ CNN Architectures
- ✓ Transfer Learning & Fine-Tuning
- ✓ Full pipeline integration

1. What is the most time-consuming stage of a deep learning project?
 - a) Training
 - b) Data collection and cleaning
 - c) Deployment

2. Why is normalization important before training?
 - a) Makes training faster and more stable
 - b) Adds noise
 - c) Reduces dataset size

3. What is data augmentation used for?
 - a) Reduce accuracy
 - b) Improve generalization
 - c) Label correction

1. What is the most time-consuming stage of a deep learning project?
 - a) Training
 - ☒ b) Data collection and cleaning
 - c) Deployment

2. Why is normalization important before training?
 - ☒ a) Makes training faster and more stable
 - b) Adds noise
 - c) Reduces dataset size

3. What is data augmentation used for?
 - a) Reduce accuracy
 - ☒ b) Improve generalization
 - c) Label correction

End-to-End Deep Learning Workflow



(Q&A)

Parham Kebria



parhamkebria@ieee.org